Change Impact Assessment Module (CIAM)

Change Impact Assessment Module (CIAM)

OOF® Origin Open Foundation®

Independent Methodological Authority

Architecture: SLGA™ — Software Lifecycle Governance Architecture

Parent Standard: Software Change Management Standard (SCMS)

Category: Governance & Enforcement

Subcategory: Software Change Management Governance

Type: Core Module

Governed Space: Software Change Impact Governance

Module ID: SLGA-SCMS-CIAM-02

Origin ID: OOF-SLGA-SCMS-CIAM

Version: 1.0

Status: Canonical Open

Effective Date: 14.10.2026

Compatibility: Methodology OS — Modular Compatibility

GIE-Compatible: Yes

AI-Readable: Yes

Authority: OOF® — OriginOpen® Foundation

Protection: Protected Methodological Work

Canonical Language: English

Canonical Definition

Change Impact Assessment Module (CIAM) is the Core Module within Software Change Management Standard (SCMS) that governs how the potential and observed consequences of proposed software changes are identified, analyzed, classified, evidenced, and translated into requirements for subsequent software lifecycle governance.

CIAM establishes change impact as a multidimensional relationship between

an identifiable proposed modification, the software objects it directly affects, the dependencies and conditions through which its consequences may propagate, and the resulting implications for software functionality, quality, security, configuration, operational continuity, and lifecycle integrity.

The module distinguishes the technical size of a modification from its

governance significance, direct impact from indirect impact, anticipated consequences from verified consequences, and impact assessment from implementation authorization.

It requires material effects, dependencies, assumptions, uncertainties,

reversibility limitations, and verification needs to remain identifiable rather than being concealed within simplified change classifications or aggregate risk scores.

CIAM applies to human-proposed changes, automated modifications,

AI-generated code, dependency updates, configuration revisions, security remediation, infrastructure-related software changes, and autonomous software systems.

Its principal governance output is the Software Change Impact Profile.

Operational Role

CIAM establishes the substantive assessment layer between change request

formation and coordinated lifecycle processing.

It governs:

• Change impact assessment identity

• Proposed change interpretation

• Assessment scope

• Current-state and intended-state references

• Directly affected software objects

• Indirect impact propagation

• Functional consequences

• Dependency consequences

• Configuration and build consequences

• Quality and security consequences

• Data and interface compatibility

• Operational consequences

• Reversibility and recovery implications

• Combined and concurrent change effects

• Assessment evidence

• Assumptions and uncertainty

• Impact significance

• Feasibility considerations

• Required verification

• Lifecycle governance implications

• Material reassessment triggers

• Impact assessment conclusions

CIAM does not authorize implementation, perform technical changes, replace specialized lifecycle assessments, or establish final operational acceptance.

Operational Question

How can the direct, indirect, combined, and potentially evolving consequences of a proposed software change be identified, evaluated, and supported by evidence so that its governance significance, affected lifecycle mechanisms, uncertainty, and required controls remain visible before the change proceeds?

Position in Architecture Governance Flow

SLGA™ Governance Flow

Software Requirements → Software Creation → Software Dependencies →

Software Configuration → Software Quality → Software Security → Software Release → Software Deployment → Software Change Management → Software Retirement

Cross-cutting: Software Lifecycle Control

SCMS Core Module Flow

Change Request Management Module (CRMM) → Change Impact Assessment Module

(CIAM) → Change Coordination Control Module (CHCM) → Lifecycle Re-entry Control Module (LRECM) → Change Closure Management Module (CCLM)

CIAM is the second Core Module within SCMS.

It receives identifiable change requests and supporting evidence from CRMM.

It evaluates the consequences of the proposed modification and establishes

the impact references required for coordination, lifecycle re-entry, and subsequent verification.

Its conclusions may be revised when implementation evidence, dependency conditions, or operational observations reveal additional consequences.

• Governed Space

Software Change Impact Governance

This governed space concerns the identification, evaluation,

classification, and evidence-supported interpretation of software change consequences.

It begins when an identifiable software change requires substantive assessment of its potential effects.

It includes direct modifications, indirect dependency effects, interactions

with other changes, and consequences extending beyond the initially proposed scope.

It continues through assessment, clarification, impact classification, uncertainty treatment, reassessment, and the establishment of applicable lifecycle governance implications.

The governed space does not end merely because an initial impact report has been produced.

Material new evidence may require the impact assessment to be reopened.

Governed Object

The primary Governed Object is:

A Software Change Impact Assessment representing the evidence-supported

relationship between an identifiable proposed software change, its current and intended software conditions, directly and indirectly affected objects, material consequences, assumptions, uncertainties, significance, feasibility considerations, and required lifecycle governance responses.

The governed object must preserve, where applicable:

Impact Assessment ID

Change Request ID

Software Product ID

Assessment Scope

Current-State Reference

Intended-State Reference

Directly Affected Objects

Indirectly Affected Objects

Dependency References

Impact Dimensions

Impact Evidence

Assumptions

Uncertainty

Impact Significance

Feasibility Conditions

Required Verification

Lifecycle Implications

Assessment Status

Assessment Version

Scope

CIAM governs:

Change Impact Recognition

Assessment Boundary Definition

Current–Intended State Comparison

Direct Impact Identification

Indirect Impact Propagation

Dependency Impact

Configuration Impact

Quality Impact

Security Impact

Data Impact

Interface Impact

Operational Impact

Resource Impact

Reversibility Assessment

Recovery Implications

Concurrent Change Impact

Impact Evidence

Impact Significance

Impact Uncertainty

Verification Requirements

Lifecycle Implications

Impact Reassessment

CIAM does not prescribe a particular risk-scoring system, static analysis tool, dependency scanner, testing framework, or software architecture methodology.

Foundational Principle

The governance significance of a software change is determined by the consequences it may introduce, not by the apparent size, simplicity, or speed of the modification.

A one-line code change may alter authorization behavior.

A dependency update may affect multiple services.

A configuration modification may create data integrity consequences.

A small firmware change may alter physical system behavior.

An automated correction may resolve one defect while introducing another.

Modification Size ≠ Impact Significance.

Local Correctness ≠ System-Level Compatibility.

Known Direct Impact ≠ Complete Change Impact.

Impact Assessment Identity

Every governed impact assessment must have a stable identity.

Relevant elements include:

Impact Assessment ID

Change Request ID

Software Product ID

Assessment Actor or Function

Assessment Time

Assessment Scope

Evidence Reference

Assessment Status

Assessment Version

Material reassessments must preserve their relationship to previous assessment versions.

Change Request Reference

CIAM must preserve the change request on which the assessment is based.

Relevant elements include:

Change Request ID

Request Version

Change Purpose

Proposed Modification

Affected Software Objects

Preliminary Scope

Supporting Evidence

Known Constraints

Known Assumptions

Request Classification

A material change to the request may invalidate part or all of an earlier impact assessment.

Assessment Purpose

The assessment purpose identifies the governance decisions and lifecycle

implications that the analysis must support.

Potential purposes include:

Determine Material Consequences

Identify Affected Components

Establish Dependency Effects

Identify Verification Requirements

Evaluate Operational Exposure

Identify Reversibility Limitations

Support Change Coordination

Support Lifecycle Re-entry

Identify Required Specialized Assessments

Establish Reassessment Conditions

The purpose must remain distinguishable from approval of the proposed change.

Assessment Scope

CIAM must define the scope of the impact assessment.

Relevant dimensions include:

Software Objects

Functional Behavior

Dependencies

Configuration

Build Conditions

Security

Quality

Data

Interfaces

Operational Environments

External Systems

Physical-System Effects Where Applicable

The assessment scope must be justified by the proposed change and available evidence.

Assessment Boundary

The assessment boundary identifies which consequences are evaluated directly and which require referral to other governance mechanisms.

Relevant boundaries include:

Software Object Boundary

Dependency Boundary

Data Boundary

Infrastructure Boundary

Security Boundary

Operational Boundary

Organizational Boundary

External System Boundary

Physical-World Boundary

A defined assessment boundary must not be used to dismiss material effects that extend beyond it.

Such effects must be recorded and referred to an applicable governance mechanism.

Current-State Baseline

CIAM must establish a sufficiently reliable reference to the software state before the proposed modification.

Relevant elements include:

Current Software Version

Current Artifact Identity

Current Configuration

Current Dependencies

Current Functional Behavior

Current Quality Conditions

Current Security Conditions

Current Data Structures

Current Operational Conditions

Known Restrictions

Where the current state cannot be sufficiently established, the limitation must remain visible.

Intended-State Reference

CIAM must identify the software state the change is expected to establish.

Relevant elements include:

Target Functionality

Target Behavior

Target Artifact

Target Configuration

Target Dependencies

Target Security Conditions

Target Quality Conditions

Target Data Compatibility

Target Operational Outcome

Expected Verification Conditions

The intended state is a proposed target, not evidence of implementation success.

Current–Intended State Comparison

CIAM must compare relevant current and intended conditions.

The comparison may include:

Existing Functionality ↔ Proposed Functionality

Existing Artifacts ↔ Proposed Artifacts

Existing Dependencies ↔ Proposed Dependencies

Existing Configuration ↔ Proposed Configuration

Existing Security Conditions ↔ Proposed Security Conditions

Existing Data Structures ↔ Proposed Data Structures

Existing Operational Behavior ↔ Expected Behavior

The comparison establishes the initial change surface.

It does not establish that all indirect consequences have been discovered.

Direct Impact

Direct impact concerns software objects and conditions intentionally

modified by the proposed change.

Examples include:

Source Code Modification

Dependency Replacement

Configuration Revision

Interface Modification

Database Schema Change

Firmware Update

Authentication Logic Change

Automation Policy Modification

Direct impact must be associated with identifiable software objects and the proposed modification.

Indirect Impact

Indirect impact concerns consequences arising through relationships with other objects, systems, or conditions.

Examples include:

Dependent Service Failure

Transitive Dependency Incompatibility

Changed Data Consumer Behavior

Altered Security Exposure

Performance Degradation

Infrastructure Capacity Effects

Unexpected Operational Interaction

Recovery Incompatibility

Indirect impact may extend beyond the software component initially modified.

Impact Propagation

CIAM must evaluate how a change may propagate through connected software relationships.

Potential propagation paths include:

Component → Dependent Component

Library → Application

Interface → External Consumer

Configuration → Runtime Behavior

Database Schema → Data Consumer

Security Policy → Access Behavior

Firmware → Device Operation

Agent Tool Configuration → Downstream Actions

Propagation assessment must distinguish supported relationships from hypothetical possibilities.

Impact Propagation Boundary

CIAM must identify where impact propagation is sufficiently understood and where material uncertainty remains.

Relevant elements include:

Known Dependency Relationships

Known Consumers

Known Integration Points

Observed Runtime Connections

Unmapped Dependencies

External System Boundaries

Unavailable Evidence

Unverified Propagation Paths

An incomplete dependency map must not be treated as proof that no additional impact exists.

Functional Impact

Functional impact concerns changes to software capabilities and behavior.

Relevant dimensions include:

Introduced Functionality

Removed Functionality

Modified Behavior

Changed Business Logic

Changed Error Handling

Changed User Interaction

Changed Automation Behavior

Changed System Outputs

CIAM must identify whether the intended functional modification may affect other established requirements.

Requirements Impact

Software changes may alter the validity or applicability of existing requirements.

Relevant elements include:

Affected Requirement IDs

Changed Functional Requirements

Changed Nonfunctional Requirements

New Requirements

Obsolete Requirements

Conflicting Requirements

Requirements Traceability

Required Reassessment

SRS governs the applicable requirements mechanisms.

CIAM identifies their change-related impact.

Software Architecture Impact

Changes may affect software structure and component relationships.

Relevant elements include:

Component Boundaries

Service Responsibilities

Interface Contracts

Data Flows

Execution Paths

Shared Infrastructure Dependencies

Architectural Assumptions

System Integration Conditions

An apparently local modification may affect architectural relationships outside its immediate implementation scope.

Dependency Impact

CIAM must evaluate material consequences for software dependencies.

Relevant elements include:

Introduced Dependencies

Removed Dependencies

Version Changes

Transitive Dependencies

Compatibility Requirements

Dependency Availability

Security Exposure

Maintenance Continuity

External Service Dependencies

SDS governs dependency identity, suitability, introduction, and continuity.

CIAM establishes which dependency governance activities are implicated by the change.

Configuration Impact

Configuration impact concerns changes to settings, baselines, and

environment-dependent behavior.

Relevant elements include:

Configuration Baseline

Runtime Parameters

Feature Flags

Environment Variables

Build Configuration

Service Configuration

Access Configuration

Infrastructure Configuration

Configuration Compatibility

Configuration impact must remain distinguishable from source code impact.

Build and Artifact Impact

CIAM must identify whether the proposed change affects software build conditions or artifact identity.

Relevant elements include:

Build Inputs

Build Dependencies

Compiler or Toolchain

Artifact Composition

Artifact Version

Reproducibility

Artifact Integrity

Distribution Requirements

Compatibility With Existing Releases

SCFS governs the relevant build and configuration mechanisms.

Quality Impact

Quality impact concerns potential effects on software quality requirements

and conditions.

Relevant dimensions include:

Functional Correctness

Reliability

Performance

Availability

Maintainability

Compatibility

Testability

Regression Exposure

Known Defects

CIAM identifies required quality reassessment.

SQS governs the applicable quality verification mechanisms.

Test Impact

CIAM must identify how the proposed change affects verification coverage.

Relevant elements include:

Existing Test Applicability

Required New Tests

Regression Test Requirements

Integration Test Requirements

Security Test Requirements

Performance Test Requirements

Environment-Specific Tests

Unverified Conditions

Evidence Reuse Limitations

Existing tests must not automatically be treated as sufficient for a materially changed software state.

Security Impact

Security impact concerns potential changes to security requirements,

exposure, and controls.

Relevant elements include:

Authentication

Authorization

Access Permissions

Attack Surface

Data Exposure

Dependency Vulnerabilities

Security Configuration

Secrets and Credentials

Auditability

Security Monitoring

Remediation Requirements

A security-motivated change may itself introduce new security consequences.

Data Impact

CIAM must evaluate potential effects on data structures, integrity, availability, and compatibility.

Relevant dimensions include:

Database Schema

Data Format

Data Migration

Data Retention

Data Integrity

Data Access

Data Consumers

Backup Compatibility

Restoration Feasibility

External Data Exchange

Data impact may remain significant even when the software artifact can be restored to its previous version.

Interface and Integration Impact

Changes may affect interfaces between software components or external systems.

Relevant elements include:

API Contracts

Message Formats

Communication Protocols

Data Exchange Semantics

Authentication Interfaces

Version Compatibility

External Consumers

Integration Timing

Failure Handling

An interface change may affect systems outside the direct control of the change requester.

Operational Impact

Operational impact concerns potential effects on software operating

conditions.

Relevant dimensions include:

Service Availability

Service Continuity

Runtime Behavior

Infrastructure Requirements

Deployment Complexity

Monitoring Coverage

Operational Restrictions

Recovery Capability

User Disruption

External System Consequences

Operational impact must be assessed according to the actual environment in which the software operates.

Infrastructure Impact

Software changes may alter infrastructure requirements.

Relevant elements include:

Compute Resources

Memory Consumption

Storage Requirements

Network Traffic

Infrastructure Configuration

Scaling Requirements

Platform Compatibility

Availability Architecture

Resource Allocation

A software modification may be technically correct while exceeding the capacity or constraints of the intended infrastructure.

Performance and Resource Impact

CIAM must identify material changes in resource consumption and performance.

Relevant dimensions include:

Execution Time

Latency

Throughput

Memory Usage

Storage Consumption

Network Consumption

Computational Demand

Energy Demand Where Relevant

Infrastructure Cost Exposure

Performance improvements in one component may produce increased consumption elsewhere.

User and External Stakeholder Impact

Software changes may affect users and external parties.

Relevant elements include:

User Functionality

Access Conditions

Service Availability

Compatibility Requirements

Required User Actions

External Integration Changes

Contractual Dependencies

Operational Communication Needs

The assessment must distinguish direct user effects from consequences mediated through dependent services.

Physical-System Impact

Where software controls or influences physical systems, CIAM must evaluate material physical-world consequences.

Examples include:

Robotic Movement

Industrial Equipment Control

Vehicle Behavior

Sensor Interpretation

Actuator Commands

Physical Safety Conditions

Equipment Availability

Emergency Stop Behavior

Software verification alone may be insufficient to establish acceptable physical-system behavior.

Autonomous System Impact

Changes affecting autonomous systems may alter decision-making or execution behavior.

Relevant dimensions include:

Agent Objectives

Decision Logic

Tool Selection

Permission Interpretation

Delegation Boundaries

Memory Access

External Actions

Human Override

Escalation Behavior

Operational Autonomy

A change to an agent's instructions or tool configuration may have consequences beyond the apparent size of the modification.

Authority and Permission Impact

CIAM must identify whether a proposed change affects legitimate authority or permissions.

Relevant elements include:

Actor Permissions

Service Accounts

Agent Tool Access

Delegation Conditions

Execution Boundaries

Approval Requirements

Human Intervention Rights

External System Access

Audit Requirements

Changes to authority-related software behavior require appropriate

governance references beyond technical implementation.

Reversibility Impact

CIAM must evaluate whether the consequences of the change can be reversed.

Relevant elements include:

Artifact Reversion

Configuration Reversion

Dependency Reversion

Data Reversion

Schema Compatibility

External Side Effects

Infrastructure State

Physical-System Consequences

A change may be reversible at the source code level while remaining irreversible at the operational level.

Rollback and Recovery Impact

CIAM must identify consequences affecting future rollback and recovery.

Relevant considerations include:

Previous Artifact Compatibility

Data Restoration Requirements

Backup Applicability

Dependency Availability

Configuration Compatibility

Recovery Execution Complexity

Potential Service Interruption

Recovery Verification

Irreversible Consequences

SDPS and its Rollback & Recovery Module govern the applicable deployment

recovery mechanisms.

CIAM identifies the change-related implications.

Concurrent Change Impact

Multiple proposed or active changes may interact.

Relevant elements include:

Shared Software Objects

Overlapping Dependencies

Concurrent Configuration Changes

Competing Interface Changes

Shared Data Structures

Release Coordination

Deployment Sequencing

Combined Verification Requirements

CIAM must not assume that individually acceptable changes remain acceptable when combined.

Cumulative Change Impact

A sequence of individually limited changes may create a material cumulative effect.

Examples include:

Progressive Dependency Drift

Accumulated Configuration Changes

Repeated Permission Expansion

Gradual Performance Degradation

Increasing Integration Complexity

Erosion of Recovery Compatibility

Accumulated Technical Constraints

Cumulative impact must be considered where the assessment evidence indicates that prior changes materially affect the proposed modification.

Change Interaction Assessment

CIAM must evaluate material interactions among changes.

Relevant relationships include:

Compatible

Conditionally Compatible

Dependent

Conflicting

Sequence-Sensitive

Unverified

Unknown

An interaction may require coordinated implementation, additional testing, or a revised lifecycle pathway.

Impact Feasibility

Impact feasibility concerns whether the proposed change can be implemented and verified under applicable conditions.

Relevant factors include:

Technical Capability

Required Expertise

Required Resources

Dependency Availability

Environment Compatibility

Verification Capability

Implementation Constraints

Operational Restrictions

Recovery Feasibility

External Dependencies

Feasibility must remain distinct from implementation authorization.

Alternative Change Strategies

Where material impact warrants comparison, CIAM may assess alternative approaches.

Potential alternatives include:

Proposed Modification

Different Technical Solution

Configuration Adjustment

Dependency Replacement

Temporary Mitigation

Functional Restriction

Deferred Change

Software Replacement

Retirement or Migration

Alternatives must be evaluated against the underlying change purpose rather than solely against implementation convenience.

Impact Evidence

CIAM must preserve evidence supporting material impact conclusions.

Potential evidence includes:

Dependency Graphs

Source Code Analysis

Test Results

Security Assessments

Configuration Comparisons

Performance Measurements

Operational Observations

Interface Specifications

Data Migration Analysis

Historical Incident Records

Simulation Results

Expert Assessments

Evidence must remain attributable to its source and applicable assessment scope.

Evidence Applicability

Existing evidence may support impact conclusions only where its assumptions remain applicable.

Relevant considerations include:

Software Version

Artifact Identity

Configuration

Dependency State

Test Environment

Operational Conditions

Evidence Age

Assessment Scope

Known Limitations

Previously Valid Evidence ≠ Automatically Applicable Evidence.

Impact Assumptions

CIAM must identify assumptions materially influencing the assessment.

Examples include:

Assumed Dependency Compatibility

Assumed Deployment Environment

Assumed Data State

Assumed User Behavior

Assumed Infrastructure Capacity

Assumed Recovery Capability

Assumed External Service Availability

Assumed Agent Behavior

Assumptions must not be presented as verified conditions.

Impact Uncertainty

CIAM must preserve uncertainty that materially affects assessment conclusions.

Potential uncertainty categories include:

Unknown Dependency

Incomplete Software Inventory

Unverified Interface Consumer

Insufficient Test Coverage

Unknown Operational Condition

Unverified Data Compatibility

Uncertain Recovery Feasibility

Conflicting Evidence

Unknown Physical-System Consequence

An absence of identified impact must not be treated as proof of impact absence when assessment coverage is insufficient.

Impact Coverage

Impact coverage concerns whether the assessment has examined the material

dimensions relevant to the proposed change.

Relevant elements include:

Known Affected Objects Covered

Known Dependencies Covered

Relevant Interfaces Covered

Security Implications Covered

Quality Implications Covered

Operational Conditions Covered

Recovery Implications Covered

Material External Effects Covered

Coverage must be assessed relative to the change's actual context.

Impact Significance

CIAM must classify material consequences according to justified assessment criteria.

Relevant factors include:

Severity

Likelihood Where Assessable

Affected Scope

Operational Criticality

Security Consequences

Data Consequences

Reversibility

Recovery Complexity

Uncertainty

Verification Difficulty

A numerical score may support classification but must not replace the underlying evidence and reasoning.

Impact Classification

CIAM may classify impact using the following governance states:

Impact Identified

Impact Under Assessment

Limited Impact Supported

Material Impact Identified

Critical Impact Identified

Indirect Impact Identified

Combined Impact Identified

Impact Conditionally Acceptable

Impact Unacceptable Under Current Conditions

Assessment Incomplete

Evidence Insufficient

Impact Conflicted

Reassessment Required

Unknown

Classification must preserve its assessment basis and scope.

Verification Implications

CIAM must identify the verification activities materially required by the change.

Potential requirements include:

Requirements Verification

Functional Testing

Regression Testing

Integration Testing

Security Testing

Performance Testing

Configuration Verification

Dependency Compatibility Testing

Data Integrity Verification

Operational Verification

Physical-System Validation Where Applicable

CIAM identifies the verification implications.

The applicable specialized standards govern their execution and acceptance.

Lifecycle Governance Implications

CIAM must identify which SLGA™ governance mechanisms may require renewed or additional activity.

Potential implications include:

Requirements Reassessment

Software Creation

Dependency Assessment

Configuration Revision

Quality Verification

Security Verification

Release Reassessment

Deployment Reassessment

Recovery Planning

Retirement or Migration Assessment

These implications provide input to Lifecycle Re-entry Control Module

(LRECM).

CIAM does not independently establish completion of lifecycle re-entry.

Impact Reassessment Triggers

Impact reassessment may be required when:

Change Scope Expands

Proposed Solution Changes

New Dependencies Are Identified

Implementation Deviates

Verification Fails

Security Findings Emerge

Operational Conditions Change

Concurrent Changes Introduce New Interactions

Recovery Feasibility Changes

New Evidence Contradicts Earlier Conclusions

Reassessment must preserve earlier assessment versions and identify which conclusions remain applicable.

Assessment Continuity

CIAM must preserve the continuity of impact assessment through change evolution.

Relevant elements include:

Original Assessment

Revised Assessment

Changed Assumptions

Newly Identified Consequences

Resolved Uncertainty

New Uncertainty

Changed Significance

Changed Verification Requirements

Changed Lifecycle Implications

Historical assessment conclusions must not be silently overwritten.

Governance Requirements

A conforming CIAM implementation must satisfy the following requirements. Assessment Identity and Scope

• Establish a stable impact assessment identity

• Associate the assessment with an identifiable change request

• Preserve the applicable request version

• Establish assessment purpose and scope

• Identify the current software state

• Identify the intended software state

• Identify assessment boundaries

• Preserve assessment limitations

Direct and Indirect Consequences

• Identify directly affected software objects

• Evaluate material indirect effects

• Assess known dependency propagation

• Identify configuration and artifact implications

• Evaluate functional consequences

• Evaluate quality consequences

• Evaluate security consequences

• Evaluate data and interface consequences

• Evaluate operational consequences

• Identify physical-system consequences where applicable

Interaction and Feasibility

• Identify relevant concurrent changes

• Evaluate material change interactions

• Consider cumulative impact where justified

• Evaluate implementation feasibility

• Identify reversibility limitations

• Identify rollback and recovery implications

• Preserve material alternatives where relevant

• Identify resource and capability constraints

Evidence and Classification

• Preserve impact evidence

• Identify evidence provenance

• Evaluate evidence applicability

• Preserve material assumptions

• Preserve uncertainty

• Evaluate assessment coverage

• Establish impact significance

• Preserve the basis for impact classification

• Distinguish assessed consequences from verified outcomes

Lifecycle and Continuity

• Identify required verification activities

• Identify affected lifecycle governance mechanisms

• Preserve applicable specialized assessment references

• Establish reassessment triggers

• Preserve material assessment revisions

• Provide structured impact outputs to CHCM and LRECM

• Prevent impact assessment conclusions from being treated as implementation authorization.

Governance Logic

Change Request → Assessment Identity → Current-State Baseline →

Intended-State Reference → Direct Change Surface → Dependency Relationships → Indirect Impact Propagation → Functional Impact → Configuration and Artifact Impact → Quality Impact → Security Impact → Data and Interface Impact → Operational Impact → Reversibility and Recovery Impact → Concurrent Change Interaction → Evidence Assessment → Coverage and Uncertainty → Impact Significance → Verification Requirements → Lifecycle Implications → Software Change Impact Profile

Where material impact is uncertain:

Impact Question → Evidence Gap → Additional Analysis → Updated Assessment

/ Unresolved Uncertainty

Where consequences extend beyond the initial scope:

Direct Impact → Propagation Path → Additional Affected Object → Expanded

Assessment → Revised Impact Classification

Where concurrent changes interact:

Change A + Change B → Shared Dependency or Condition → Combined Impact →

Coordination Requirement

Where new evidence changes the assessment:

Previous Impact Profile → New Evidence → Assumption Reassessment → Revised

Consequences → Updated Profile

CIAM must preserve the distinction between assessing a proposed change and authorizing its implementation.

Governance Outputs

The principal output is:

Software Change Impact Profile

Supporting outputs may include:

Impact Assessment Identity

Current–Intended State Comparison

Direct Impact Register

Indirect Impact Register

Impact Propagation Map

Dependency Impact Record

Configuration Impact Record

Quality Impact Record

Security Impact Record

Data and Interface Impact Record

Operational Impact Record

Infrastructure Impact Record

Reversibility Assessment

Recovery Implication Record

Concurrent Change Interaction Record

Impact Evidence Register

Assumption Register

Uncertainty Register

Impact Coverage Assessment

Impact Significance Classification

Verification Requirement Profile

Lifecycle Governance Implication Profile

Impact Reassessment Record

Minimum Implementation Framework

A minimum CIAM implementation must establish eight governance layers. Layer 1 — Assessment Identity

Identify the change request, applicable request version, software product,

assessment scope, and responsible assessment function. Layer 2 — State Comparison

Establish the relevant current software state, intended state, and proposed

modification. Layer 3 — Direct Impact

Identify the software objects, functions, artifacts, configurations, and

conditions directly affected. Layer 4 — Indirect Impact

Evaluate dependency propagation, interfaces, data relationships, external

systems, and other material indirect consequences. Layer 5 — Consequence Assessment

Evaluate quality, security, operational, resource, reversibility, recovery,

and combined-change implications. Layer 6 — Evidence and Uncertainty

Preserve supporting evidence, assumptions, assessment coverage,

limitations, and unresolved material questions. Layer 7 — Governance Implications

Establish impact significance, verification requirements, coordination

implications, and affected lifecycle mechanisms. Layer 8 — Governed Output and Continuity

Establish the Software Change Impact Profile, preserve reassessment

triggers, and provide structured outputs to CHCM and LRECM.

The implementation may be manual, tool-assisted, automated, or AI-supported.

The methodological requirements remain independent of the assessment

technology.

Cross-Module Boundary

CRMM → CIAM

CRMM establishes the change request identity, origin, purpose, preliminary

scope, and supporting evidence.

CIAM evaluates the substantive consequences of the proposed modification.

CIAM → CHCM

CIAM identifies affected objects, dependencies, interactions, risks, and

coordination implications.

CHCM governs the coordination of actors, decisions, activities, and related

changes.

CIAM → LRECM

CIAM identifies affected lifecycle mechanisms and required verification

implications.

LRECM determines and controls the applicable lifecycle re-entry pathway.

CIAM → CCLM

CIAM establishes expected consequences and verification implications.

CCLM uses relevant assessment references when evaluating the resulting change outcome and unresolved obligations.

Impact Assessed ≠ Change Authorized.

Cross-Architecture Boundary

CIAM identifies change-related consequences across SLGA™ but does not replace specialized lifecycle governance.

SRS — Software Requirements Standard

Requirements impact and traceability.

SCS — Software Creation Standard

Software artifact and development implications.

SDS — Software Dependency Standard

Dependency identity, suitability, compatibility, and continuity.

SCFS — Software Configuration Standard

Configuration baselines, builds, artifact identities, and reproducibility.

SQS — Software Quality Standard

Quality requirements, testing, defects, and quality assessment.

SSS — Software Security Standard

Security exposure, vulnerabilities, and remediation.

SRLS — Software Release Standard

Release readiness, authorization, and release records.

SDPS — Software Deployment Standard

Deployment conditions, production versions, monitoring, rollback, and

recovery.

SRTS — Software Retirement Standard

Migration, retirement, and residual dependency implications.

SLCS — Software Lifecycle Control Standard

Lifecycle transitions, exceptions, state consistency, and continuity.

Applicable OOF® authority, accountability, autonomous systems, and

operational reality governance references may also be required when a change affects those domains.

Practical Applications

Enterprise Software Maintenance

Assessing modifications across interconnected services, shared libraries,

and production environments.

Dependency Modernization

Identifying compatibility, security, configuration, and operational

consequences of dependency upgrades.

AI-Assisted Development

Evaluating AI-generated code changes beyond their immediate functional

correctness.

Autonomous Software Operations

Assessing how agent-generated changes may alter permissions, tool behavior, infrastructure interactions, or downstream execution.

Critical Infrastructure Software

Identifying software change consequences for service continuity, recovery

capability, and physical-system operation.

Robotics and Embedded Systems

Assessing firmware and control logic modifications that may affect sensors, actuators, or operational safety.

Use Case 1 — A One-Line Permission Change Creates System-Wide Security

Exposure

A developer proposes a one-line modification to an authorization function.

The modification is intended to resolve an access problem affecting a limited user group.

Local tests confirm that the affected users can access the required function.

However, the modified condition also grants access to other users who

previously lacked permission.

CIAM evaluates:

Change Request → Direct Code Modification → Authorization Logic → Affected

User Groups → Security Impact → Verification Requirements → Lifecycle Implications

The assessment identifies:

• The source code modification is small

• The intended functional correction is achieved in local testing

• The modification changes the effective permission boundary

• Additional users may receive unintended access

• Security assessment and broader authorization testing are required

The appropriate classification is:

Material Security Impact Identified — Additional Verification Required.

One-Line Modification ≠ Limited Governance Impact.

• Use Case 2 — A Robotic Firmware Update Alters Physical Behavior Through

an Indirect Dependency

A robotics manufacturer proposes a firmware update to improve sensor

processing efficiency.

The updated sensor library passes its component-level tests.

However, the update changes the timing characteristics of sensor data

delivered to the movement-control system.

The movement-control software depends on timing assumptions established

under the previous firmware.

CIAM evaluates:

Firmware Change → Sensor Processing → Timing Dependency → Movement Control

→ Physical-System Consequences → Verification Requirements

The assessment identifies:

• The direct change affects sensor processing

• The movement-control system is indirectly affected

• Existing timing assumptions may no longer remain valid

• Component-level testing does not establish complete system

• Integration and physical-system verification are required before the

compatibility. impact can be considered sufficiently assessed.

The appropriate classification is:

Material Indirect Impact Identified — System-Level Verification Required.

Component Test Success ≠ Physical-System Compatibility.

Critical Distinctions

Modification Size ≠ Impact Significance.

Local Correctness ≠ System-Level Compatibility.

Known Direct Impact ≠ Complete Change Impact.

Proposed State ≠ Verified Resulting State.

Directly Modified Object ≠ Only Affected Object.

Limited Change Scope ≠ Limited Operational Consequences.

Dependency Identified ≠ Dependency Impact Fully Understood.

No Known Impact ≠ Verified Absence of Impact.

Existing Test Coverage ≠ Sufficient Change Verification.

Previously Valid Evidence ≠ Automatically Applicable Evidence.

Individual Change Acceptability ≠ Combined Change Compatibility.

Source Code Reversibility ≠ Operational Reversibility.

Assessment Completed ≠ Material Uncertainty Resolved.

Impact Classification ≠ Implementation Authorization.

Feasible Change ≠ Legitimately Authorized Change.

Impact Assessed ≠ Change Authorized.

Canonical Boundary

CIAM governs the identification, assessment, classification, and

evidence-supported interpretation of the direct, indirect, combined, and evolving consequences of proposed software changes.

It establishes the relationship between a change request, current and

intended software states, affected objects, impact propagation, material consequences, assumptions, uncertainty, verification requirements, and lifecycle governance implications.

CIAM does not independently authorize implementation, execute software modifications, replace specialized quality or security verification, or establish final operational acceptance.

Its principal governance output is the:

Software Change Impact Profile

This output provides the evidence-supported basis for change coordination,

lifecycle re-entry, and subsequent change outcome assessment.

Canonical Closing Statement

A software change cannot be assessed reliably by examining only the

component being modified. Its significance depends on the relationships, conditions, and operational consequences that the modification may alter.

A one-line permission change may create system-wide security exposure.

A dependency update may affect services that the requester does not directly control.

A configuration adjustment may invalidate earlier quality evidence.

A database migration may prevent safe rollback.

An AI-generated patch may pass local tests while introducing new operational risks.

And a firmware modification may alter physical behavior through dependencies that remain invisible at the source code level.

CIAM establishes the governance structure required to identify and assess

these consequences.

It requires change impact to be evaluated across relevant software objects,

dependencies, interfaces, quality conditions, security requirements, operational environments, and recovery constraints.

It preserves direct and indirect effects, evidence limitations, material

assumptions, and unresolved uncertainty.

It recognizes that individually acceptable changes may produce unacceptable combined consequences.

It identifies the lifecycle governance mechanisms that must be revisited without replacing their specialized responsibilities.

And it ensures that impact conclusions remain traceable and subject to reassessment when new evidence or changing conditions invalidate earlier assumptions.

The purpose of software change impact assessment is not merely to predict whether a modification will work. It is to establish what the modification may affect, how its consequences may propagate, which conditions require verification, and which governance obligations must be satisfied before the resulting software state can be considered acceptable.

Change Impact Assessment Module (CIAM) transforms software change analysis

from a local technical estimate into an evidence-supported governance assessment of system-wide consequences, uncertainty, and lifecycle obligations.